МИНОБРНАУКИ РОССИИ
РТУ МИРЭА
Институт перспективных технологий
и индустриального программирования
Кафедра индустриального программирования

Практическое занятие № 2

Симулятор работы лифтов в здании

Тема 15
Выбор темы и сбор требований
Спецификация требований SRS и UML-диаграммы
Дисциплина «Технологии индустриального программирования»

Выполнил
Исмаилов Полад Вели оглы
Группа ЭФБО-04-25

Преподаватель
Афонин Сергей Александрович

Москва 2026

1 Введение

1.1 Цель

Разработать учебный консольный симулятор, в котором несколько лифтов одновременно обслуживают вызовы пассажиров. Модель позволяет наблюдать распределение заявок, движение кабин и время ожидания.

1.2 Область действия

Проект реализуется на C++. Каждый лифт обслуживается отдельным рабочим потоком. Пользователь задаёт параметры здания, создаёт заявки, наблюдает состояние системы и получает итоговую статистику.

Заявка содержит этаж отправления и этаж назначения. Эти данные вводятся заранее: моделируется система назначения лифта по пункту назначения. Каждый лифт перевозит одного условного пассажира за рейс. Объединение пассажиров в кабине не входит в первую версию.

1.3 Границы модели

Симулятор не управляет реальным оборудованием. В первой версии не моделируются масса пассажиров, перегрузка, поломки, аварийное питание и физика привода. Приостановка и ускорение времени также не предусмотрены. Скорость движения задаётся до запуска.

1.4 Определения и сокращения

SRS — спецификация требований к программному обеспечению.
UML — язык визуального моделирования.
Заявка — поездка одного условного пассажира между двумя этажами.
Диспетчер — компонент назначения заявок лифтам.
Время ожидания — интервал от создания заявки до открытия дверей на этаже отправления.
Mutex — средство взаимного исключения при доступе к общим данным.

2 Общее описание и сбор требований

2.1 Описание продукта

Автономная консольная программа для учебных экспериментов. Основные функции: настройка здания, запуск моделирования, добавление поездок, просмотр состояния лифтов, завершение работы и вывод статистики.

2.2 Пользователи

Оператор — студент или преподаватель, который запускает сценарии и оценивает работу алгоритма. От пользователя требуется умение вводить команды в консоли. Пассажир представлен данными заявки, а не отдельным пользователем приложения.

2.3 Методология сбора требований

Использованы анализ задания и User Stories. Источники: выданное практическое задание и перечень тем проектов. Из них получены обязательные условия: C++, многопоточная модель, несколько лифтов и обработка вызовов с этажей.

Далее сформулированы пользовательские сценарии, выделены функции и ограничения, назначены приоритеты. Реальное интервью с заказчиком не проводилось. Числовые пределы, правила диспетчеризации и упрощения ниже являются принятыми проектными решениями, а не цитатами заказчика.

2.4 Принятые условия

Здание содержит от 2 до 30 этажей и от 2 до 8 лифтов. Этажи нумеруются с 1. В начале все кабины свободны и находятся на первом этаже. Время перемещения на этаж: 100–5000 мс; остановки с открытыми дверями: 100–5000 мс. Значения по умолчанию: 10 этажей, 3 лифта, 1000 и 500 мс соответственно.

3 Иерархия требований по Вигерсу

3.1 Бизнес-требования

BR-01. Предоставить учебную среду для демонстрации параллельной работы нескольких лифтов.
BR-02. Обеспечить оценку обслуживания по количеству завершённых поездок и времени ожидания.
BR-03. Позволить проводить эксперименты без подключения реального лифтового оборудования.

3.2 Пользовательские требования

UR-01. Как оператор, я хочу задать число этажей и лифтов, чтобы исследовать разные здания.
UR-02. Как оператор, я хочу добавлять поездки во время работы модели, чтобы имитировать поток пассажиров.
UR-03. Как оператор, я хочу видеть этаж, состояние и текущую заявку каждого лифта, чтобы следить за выполнением поездок.
UR-04. Как оператор, я хочу получить статистику, чтобы оценить ожидание пассажиров.
UR-05. Как оператор, я хочу корректно завершить моделирование, чтобы принятые заявки не были потеряны.

3.3 Связь уровней требований

BR-01 → UR-01, UR-02, UR-03 → FR-01–FR-06.
BR-02 → UR-04 → FR-07.
BR-03 → UR-01 → FR-01, NFR-01.
UR-05 → FR-08, NFR-02.

Функциональные требования определяют действия системы. Нефункциональные требования задают условия реализации, надёжности и проверки. Их спецификации приведены на следующей странице.

4 Спецификация требований

Функциональные требования

FR-01. До запуска проверять параметры здания и времени. При ошибке выводить причину; моделирование не запускать.
FR-02. Принимать заявку с разными этажами в пределах здания. Присваивать уникальный ID и фиксировать время поступления. Неверные заявки отклонять.
FR-03. Хранить ожидающие заявки в порядке поступления. Старейшую заявку назначать ближайшему свободному лифту; при равном расстоянии выбирать меньший ID лифта. Если свободных кабин нет, оставлять заявку в очереди.
FR-04. Выполнять поездку по шагам: движение к пассажиру, открытие и закрытие дверей, движение к назначению, открытие и закрытие дверей, завершение заявки. При открытых дверях движение запрещено.
FR-05. Показывать состояние кабин: свободен, едет к пассажиру, посадка, едет к назначению, высадка. Показывать этаж и ID заявки.
FR-06. Записывать события в консольный журнал: время, ID лифта, ID заявки и тип события.
FR-07. Выводить число принятых и завершённых заявок, размер очереди, число активных поездок, среднее и максимальное ожидание среди заявок с состоявшейся посадкой. Если посадок нет, выводить «нет данных».
FR-08. По команде stop прекратить приём новых заявок, обслужить все принятые, завершить рабочие потоки и вывести итоговую статистику.
FR-09. Команда start после настройки создаёт потоки и запускает модель. Повторный start во время работы отклоняется; до запуска заявки не принимаются.

Нефункциональные требования

NFR-01. Использовать C++17 и стандартную библиотеку; не требовать сетевых сервисов.
NFR-02. Каждый лифт работает в отдельном std::thread. Общая очередь, состояния, статистика и журнал защищены от гонок. Потоки без работы ожидают условную переменную, без активного опроса.
NFR-03. В тесте из 1000 корректных заявок каждая обслуживается ровно один раз; после прекращения приёма заявок программа завершается в пределах времени, указанного в AT-07.
NFR-04. Неверные команды не завершают программу. Сообщение содержит причину ошибки.
NFR-05. Длительности измеряются монотонными часами. Планировщик ОС может увеличивать фактическое время; работа в реальном времени не гарантируется.

5 Бэклог и критерии приёмки

5.1 Приоритеты

Обязательно: P1 — настройка и запуск (FR-01, FR-09); P2 — заявки и очередь (FR-02); P3 — диспетчер и потоки (FR-03, FR-04); P4 — наблюдение (FR-05, FR-06); P5 — статистика и остановка (FR-07, FR-08).
Порядок реализации: P1 → P2 → P3 → P4 → P5.
Возможное развитие после первой версии: пакетный генератор вызовов и сравнение алгоритмов назначения.

5.2 Проверки приёмки

AT-01 / FR-01, FR-02. При 10 этажах заявка 2 → 7 принята; 0 → 7, 2 → 11 и 4 → 4 отклонены. Некорректная настройка не запускает модель.
AT-02 / FR-03. При свободных лифтах на этажах 1 и 8 вызов с 6 назначается второму. При одинаковых расстояниях выбирается меньший ID.
AT-03 / FR-03, FR-04. Для двух лифтов и трёх заявок первые две назначаются свободным кабинам, третья ждёт освобождения. Обе кабины движутся параллельно.
AT-04 / FR-04–FR-06. Для поездки 2 → 7 журнал содержит посадку на 2 и высадку на 7. Между открытием и закрытием дверей этаж не меняется.
AT-05 / FR-07. При ожиданиях 2, 4 и 6 секунд среднее равно 4, максимум — 6. При отсутствии посадок отображается «нет данных».
AT-06 / FR-08. После stop новые заявки отклоняются; очередь и активные поездки завершаются, все потоки присоединяются.
AT-07 / NFR-02, NFR-03. Здание: 2 этажа, 2 лифта; движение и остановка — по 100 мс. До запуска рабочих потоков тест подаёт 1000 заявок 1 → 2 с ID 1–1000. Приём закрывается, потоки запускаются. За 600 с должны завершиться ровно эти 1000 заявок, без повторов. ThreadSanitizer не должен сообщать о гонках данных.

Тесты описывают приёмку будущей программы; результаты выполнения пока отсутствуют.

5 Бэклог и критерии приёмки

5.3 Дополнительные проверки

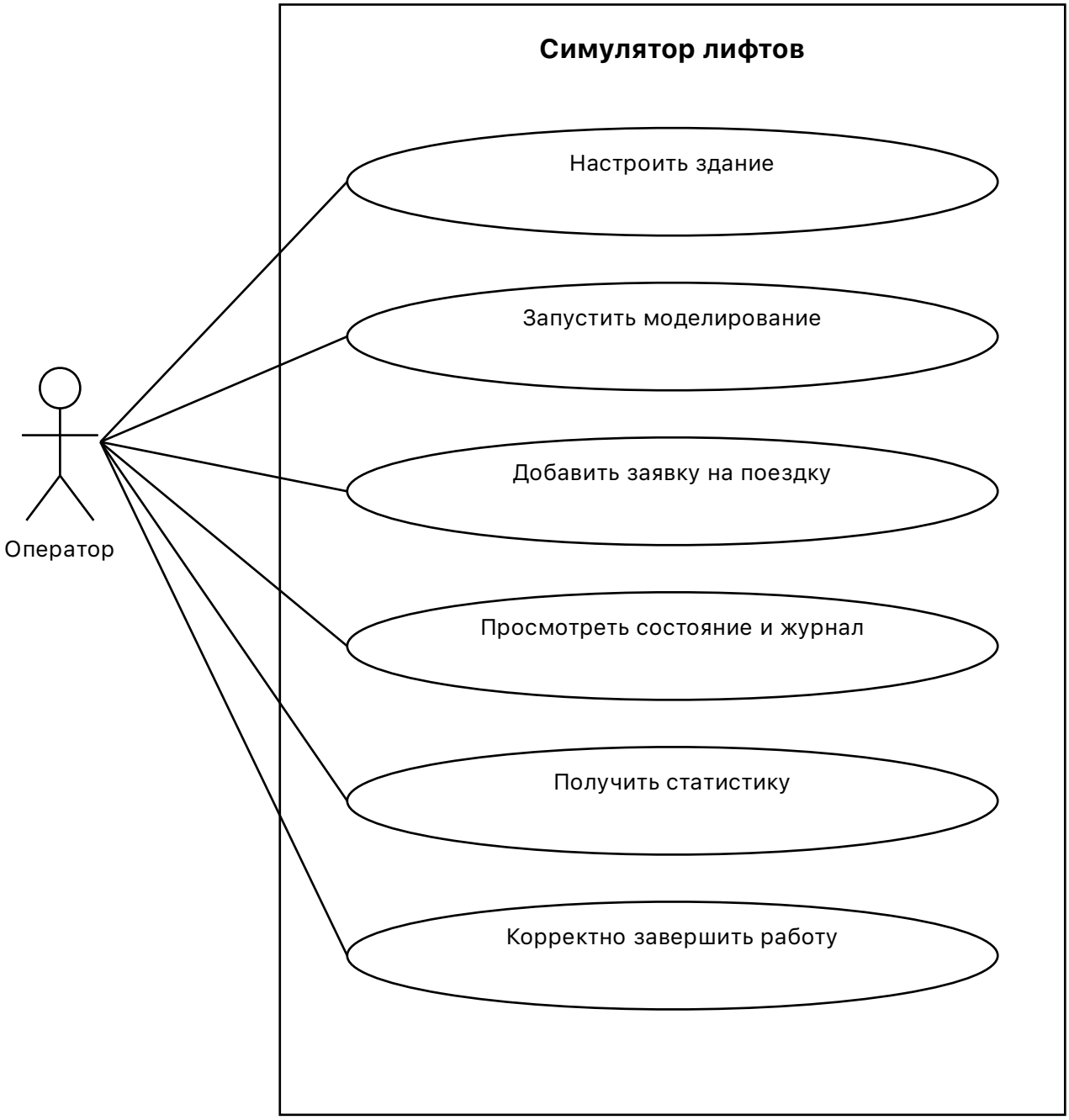
AT-08 / FR-01. Настройки 2 и 30 этажей, 2 и 8 лифтов, 100 и 5000 мс принимаются. Значения 1 и 31 для этажей, 1 и 9 для лифтов, 99 и 5001 мс отклоняются.
AT-09 / NFR-04. Команды unknown и заявка с нечисловым этажом вызывают сообщение об ошибке; следующая корректная команда выполняется.
AT-10 / FR-03. При занятых кабинах поступают заявки A, B, C. После освобождения одного лифта назначается A, затем B и C в порядке освобождения кабин.
AT-11 / FR-05, FR-06. На каждой стадии поездки проверяются состояние, этаж и ID заявки. Каждая запись журнала содержит время, ID лифта, ID заявки и событие. Записи разных потоков не смешиваются.
AT-12 / FR-07. В согласованном снимке принято = в очереди + активных + завершено. После stop очередь и число активных равны нулю, принято = завершено.
AT-13 / FR-08, FR-09. До start заявки отклоняются. Первый start запускает модель; повторный не создаёт новых потоков. При пустой очереди stop завершает потоки за 2 с на тестовом стенде.

5.4 Проверка технических ограничений

AT-14 / NFR-01. Проект собирается компилятором с поддержкой C++17 без сторонних библиотек и выполняет поездку при отключённой сети.
AT-15 / NFR-05. При ревью кода проверяется использование std::chrono::steady_clock для интервалов. Модульный тест расчёта: создание в t=0, открытие дверей в t=2 с, закрытие в t=2,5 с — ожидание равно 2 с.
AT-16 / NFR-02. При отсутствии заявок рабочие потоки ожидают condition_variable. В критических секциях нет ожидания движения и дверей; stop уведомляет все ожидающие потоки.

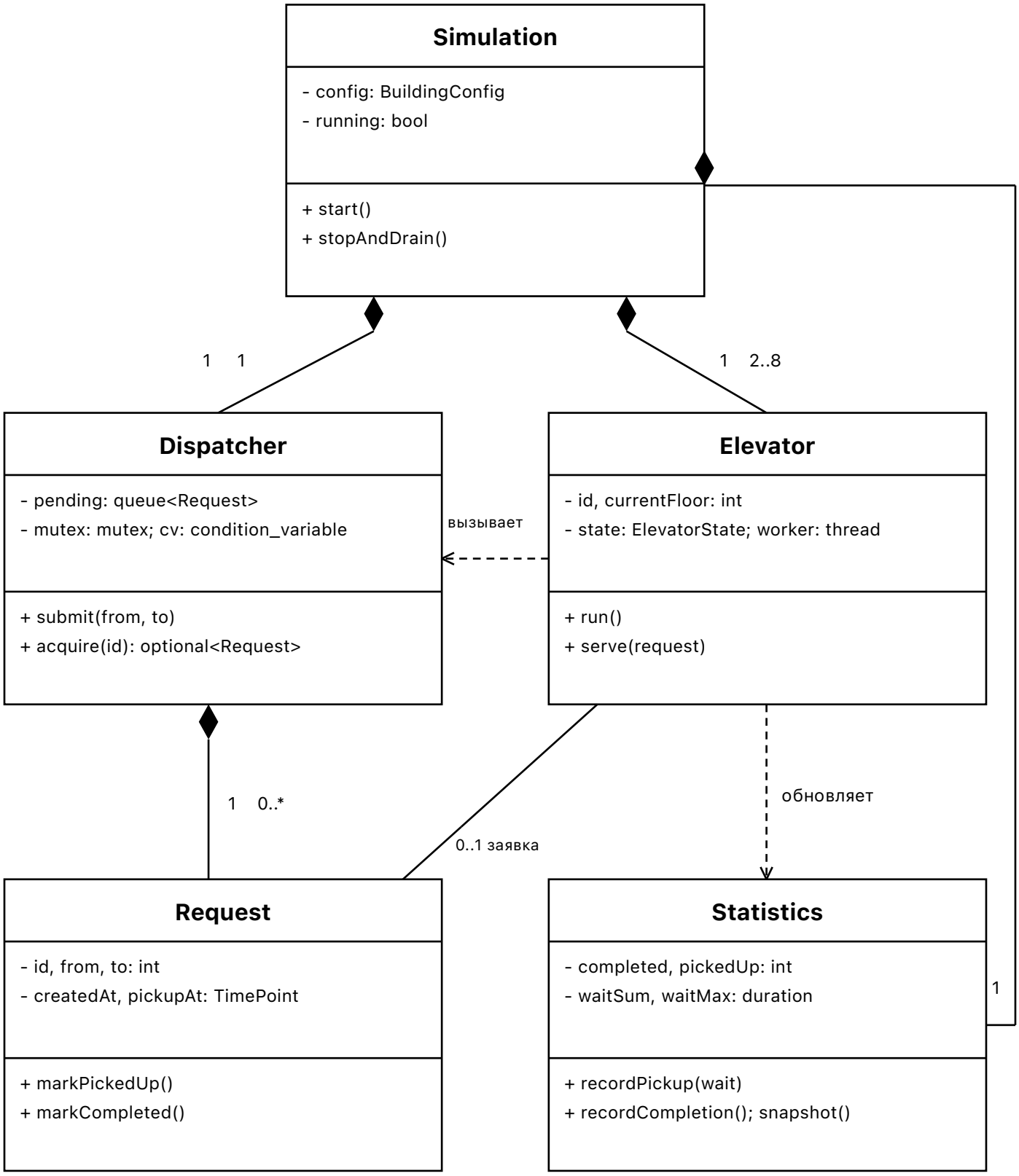
Тестовый стенд для временных критериев: 4 логических ядра, 8 ГБ ОЗУ, без другой тяжёлой нагрузки. AT-07 выполняется через тестовый интерфейс заполнения очереди, а не через пользовательские команды.

6 Диаграмма вариантов использования



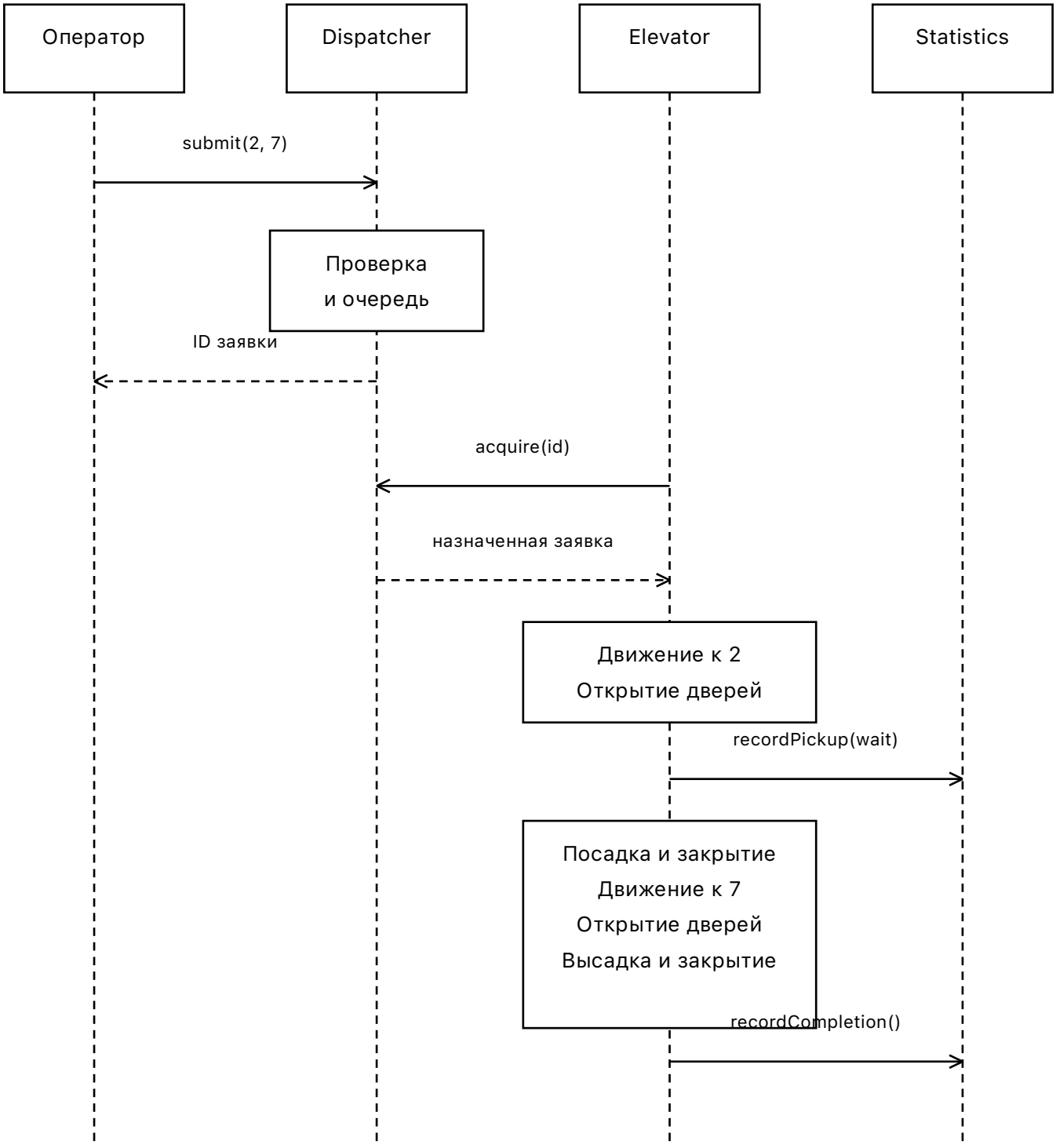
Актор — оператор консольной программы. Диспетчер и лифты входят в систему и не являются внешними акторами. Варианты использования покрывают UR-01–UR-05.

7 Диаграмма классов



◆ — композиция; пунктирная стрелка — зависимость. Simulation владеет Statistics. У лифта не более одной активной заявки. acquire() возвращает nullopt при остановке и пустой очереди. При назначении выбирается ближайший свободный лифт, а не первый вызвавший acquire().

8 Диаграмма последовательности



Основной сценарий: корректная заявка 2 → 7, свободный лифт. acquire() атомарно выбирает назначение под блокировкой. Если подходящей работы нет, поток лифта ожидает cv. Блокировка не удерживается во время движения и остановок. После рейса лифт становится свободным и снова запрашивает работу. Некорректный submit возвращает ошибку без создания заявки.